

Secure Code Review Cheat Sheet

Introduction

Secure Code Review is the process of manually examining source code to identify security vulnerabilities that automated tools often miss. It involves analyzing application logic, data flow, and implementation details to detect security flaws that require human expertise and contextual understanding.

Manual Code Review complements automated security testing tools (SAST/DAST) by focusing on areas where human analysis provides the most value, including business logic validation, complex security implementations, and context-specific vulnerabilities. While automated tools can assist by highlighting potential areas of concern, the core analysis relies on human judgment and domain expertise.

Security-Focused Review differs from functional code review by specifically targeting security concerns such as input validation, authentication mechanisms, authorization controls, cryptographic implementations, and potential attack vectors.

Review Types

Baseline Reviews examine the entire codebase comprehensively. Use for:

- New applications or major releases
- Legacy system onboarding
- Compliance requirements
- Post-incident analysis

Diff-Based Reviews focus on code changes only. Use for:

- Pull requests and commits
- Daily development workflow
- Feature completion
- Continuous security validation

This cheat sheet provides practical guidance for conducting effective manual security code reviews, with emphasis on both baseline and incremental review methodologies.

Review Methodology

Preparation

For All Reviews:

- Understand application architecture and business requirements
- Gather threat models and previous security findings
- Identify critical assets and high-risk functions
- Review security requirements and documentation

Additional for Baseline Reviews:

- Map complete application boundaries and dependencies
- Analyze overall security architecture
- Review security incident history
- Audit all third-party libraries

Additional for Diff-Based Reviews:

- Identify modified files and affected components
- Assess impact on existing security controls
- Understand purpose of changes
- Prioritize high-risk modifications

Review Process

Baseline Review Steps:

1. Architecture review for security anti-patterns
2. Entry point analysis and input validation
3. Authentication and authorization verification
4. Data flow tracing
5. Business logic analysis
6. Cryptographic implementation review
7. Error handling verification
8. Configuration and deployment review

Diff-Based Review Steps:

1. Analyze impact on existing security controls

2. Identify new attack vectors
3. Verify security at modified trust boundaries
4. Check new integrations
5. Ensure no security regression
6. Apply relevant security patterns

Common Vulnerability Patterns

Input Validation Vulnerabilities

Check for missing server-side validation, improper sanitization, and weak input filtering. For more information, see [Input Validation Cheat Sheet](#).

Injection Vulnerabilities

SQL Injection:

Look for string concatenation in database queries and unsafe query construction. For more information, see [SQL Injection Prevention Cheat Sheet](#).

Cross-Site Scripting (XSS):

Review output encoding, DOM manipulation, and user input rendering. For more information, see [Cross Site Scripting Prevention Cheat Sheet](#).

Path Traversal:

Check for unsafe file path construction and directory traversal vulnerabilities. For more information, see [File Upload Cheat Sheet](#).

Command Injection:

Identify direct command execution with user input and unsafe system calls. For more information, see [OS Command Injection Defense Cheat Sheet](#).

NoSQL Injection:

Examine NoSQL query construction and parameter binding. For more information, see [NoSQL Security Cheat Sheet](#).

Authentication & Session Management Vulnerabilities

Review authentication mechanisms, session token generation, and user credential handling. For more information, refer to [Authentication Cheat Sheet](#) and [Session Management Cheat](#)

[Sheet.](#)

Access Control Vulnerabilities

Examine authorization checks, role-based access controls, and privilege escalation prevention. For more information, see [Authorization Cheat Sheet](#).

Deserialization Vulnerabilities

Insecure Deserialization:

Check for unsafe deserialization of untrusted data and object injection vulnerabilities. For more information, see [Deserialization Cheat Sheet](#).

XML External Entity (XXE):

Review XML parsing configurations and external entity processing. For more information, see [XML External Entity Prevention Cheat Sheet](#).

Cryptographic Implementation Flaws

Examine encryption algorithms, key management, and cryptographic implementations. For more information, refer to [Cryptographic Storage Cheat Sheet](#).

Review Techniques

Code Pattern Analysis

Focus on high-risk code patterns:

- Input processing and validation functions
- Database query construction and ORM usage
- File operations and path handling
- Authentication and session management logic
- Authorization and access control checks
- Cryptographic operations and key management
- Error handling and logging mechanisms
- Configuration loading and environment variables

Data Flow Analysis

Trace data through the application:

1. **Identify Sources:** User inputs, file uploads, API calls, database reads, environment variables
2. **Follow Processing:** Validation, transformation, business logic, caching
3. **Check Sinks:** Database queries, file writes, output rendering, logging, external APIs
4. **Validate Boundaries:** Input validation and output encoding at trust boundaries
5. **Trust Zones:** Verify security controls at each trust boundary crossing
6. **Data Classification:** Ensure sensitive data receives appropriate protection

Threat-Based Review

Align review with common attack patterns:

- **OWASP Top 10:** Focus on prevalent web application risks
- **STRIDE Model:** Spoofing, Tampering, Repudiation, Information Disclosure, DoS, Elevation
- **Attack Trees:** Map potential attack paths through the application
- **Abuse Cases:** Consider how features could be misused by attackers
- **Security Controls:** Verify defense-in-depth implementation

Business Logic Review

Analyze application workflows for:

- State management and transition validation
- Race conditions and concurrency issues
- Transaction integrity and rollback mechanisms
- Resource limits and quota enforcement
- Authorization at each workflow step
- Workflow bypass opportunities

Review Checklists

Input Validation

- ☐ **Server-side validation:** All inputs validated on server regardless of client-side checks
- ☐ **Allowlist validation:** Uses allowlists rather than blocklists for input validation
- ☐ **Output encoding:** Context-appropriate encoding (HTML, JavaScript, CSS, URL, SQL)

- ☐ **File upload security:** Content-based validation, size limits, safe storage
- ☐ **SQL injection prevention:** Parameterized queries or stored procedures used
- ☐ **Length limits:** Input length restrictions enforced
- ☐ **Character handling:** Special characters and Unicode properly processed
- ☐ **Error messages:** No sensitive information disclosed in error responses

Authentication & Session Management

- ☐ **Password security:** Strong hashing algorithms and salt usage (for more information, see [Password Storage Cheat Sheet](#))
- ☐ **Account protection:** Lockout mechanisms with appropriate thresholds
- ☐ **Session management:** Secure token generation (≥ 128 bits entropy)
- ☐ **Session lifecycle:** Proper invalidation on logout/timeout
- ☐ **Re-authentication:** Required for sensitive operations
- ☐ **Multi-factor authentication:** Implementation for high-risk accounts (for more information, see [Multifactor Authentication Cheat Sheet](#))
- ☐ **Password reset:** Secure, time-limited reset mechanisms (for more information, see [Forgot Password Cheat Sheet](#))
- ☐ **Session security:** HttpOnly, Secure, SameSite cookie attributes
- ☐ **Concurrent sessions:** Appropriate limits and monitoring

Authorization

- ☐ **Server-side enforcement:** All access controls enforced server-side
- ☐ **Fail-safe defaults:** Default deny access policy
- ☐ **IDOR prevention:** Proper authorization for resource access
- ☐ **Function-level controls:** Administrative functions properly protected
- ☐ **Role validation:** Role assignments cannot be manipulated
- ☐ **Privilege escalation:** Horizontal and vertical escalation prevented
- ☐ **Centralized decisions:** Access control logic centralized
- ☐ **Post-authentication checks:** Authorization verified after authentication

Cryptography

- ☐ **Strong algorithms:** Modern algorithms (AES-256, RSA-2048+, ECDSA P-256+)

- ☐ **Key management:** Proper key generation, storage, and rotation (for more information, see [Key Management Cheat Sheet](#))
- ☐ **Certificate validation:** Proper validation including hostname verification
- ☐ **Random generation:** Cryptographically secure random number generation
- ☐ **Data protection:** Encryption at rest and in transit
- ☐ **IV/Nonce handling:** Unique and unpredictable initialization vectors
- ☐ **Library maintenance:** Up-to-date cryptographic libraries
- ☐ **Side-channel protection:** Consideration of timing and other side-channel attacks

Business Logic

- ☐ **Workflow integrity:** Proper state validation in multi-step processes
- ☐ **Race condition prevention:** Synchronization in concurrent operations
- ☐ **Transaction atomicity:** Proper rollback and consistency mechanisms
- ☐ **Resource limits:** Rate limiting and resource quotas implemented
- ☐ **Business rule enforcement:** Cannot bypass rules through direct API access

Configuration & Deployment

- ☐ **Secure defaults:** Security-focused default configurations
- ☐ **Environment separation:** Proper isolation between environments
- ☐ **Secrets management:** No hardcoded secrets, proper secret storage and rotation (for more information, see [Secrets Management Cheat Sheet](#))
- ☐ **Error handling:** Graceful error handling without information disclosure (for more information, see [Error Handling Cheat Sheet](#))
- ☐ **Logging security:** Sensitive data not logged, proper log protection (for more information, see [Logging Cheat Sheet](#))
- ☐ **Security headers:** Appropriate HTTP security headers configured (for more information, see [HTTP Headers Cheat Sheet](#))
- ☐ **TLS configuration:** Strong cipher suites and protocol versions (for more information, see [Transport Layer Security Cheat Sheet](#))
- ☐ **Dependency management:** Up-to-date libraries without known vulnerabilities (for more information, see [Vulnerable Dependency Management Cheat Sheet](#))

Security Monitoring

- ☐ **Security events:** Authentication failures, authorization violations logged

- ☐ **Anomaly detection:** Unusual patterns and behaviors monitored
- ☐ **Audit trails:** Complete audit logs for sensitive operations
- ☐ **Real-time alerts:** Critical security events trigger immediate notifications
- ☐ **Log integrity:** Logs protected from tampering and unauthorized access
- ☐ **Incident response:** Clear procedures for security incident handling

Tools and Techniques

Code Editors

Use editors with security extensions:

- Visual Studio Code with ESLint, SonarLint
- IntelliJ IDEA with SpotBugs, SonarLint
- Eclipse with security plugins
- Vim/Neovim with security linters

Command-Line Pattern Detection

```
# Find hardcoded secrets
grep -ri "password\s*=\|api_key\s*=\|secret\s*=" source/

# Find unsafe functions
grep -r "eval(\|exec(\|innerHTML\|document\.write" source/

# Find potential injections
grep -r "SELECT.*\|executeQuery.*" source/
```

Manual Review Focus Areas

Human Expertise Advantages:

- **Business Logic Flaws:** Complex workflows and state management issues that require domain understanding
- **Context-Specific Vulnerabilities:** Security issues that depend on application-specific business rules
- **Authorization Logic:** Complex permission models and access control implementations
- **Race Conditions:** Timing-based vulnerabilities in concurrent operations
- **Cryptographic Misuse:** Proper implementation of cryptographic primitives and protocols
- **Architecture Security:** High-level design flaws and security anti-patterns

Manual Analysis Techniques:

- **Code Path Tracing:** Following execution paths through complex business logic
- **State Analysis:** Understanding application state transitions and validation
- **Trust Boundary Mapping:** Identifying and analyzing security control points
- **Threat Modeling Integration:** Applying threat models to specific code implementations
- **Attack Scenario Simulation:** Mentally simulating attack paths through the code

Automated Tool Integration

Supporting Manual Reviews:

- **SAST Tool Triage:** Use automated findings to prioritize manual review areas
- **Dependency Scanning:** Identify vulnerable libraries requiring manual assessment
- **Code Quality Metrics:** Focus manual effort on complex or frequently changed code
- **Pattern Detection:** Use tools to highlight potential security anti-patterns for human analysis

Tool Integration Strategy:

- **Pre-Review Scanning:** Run automated tools before manual review to identify obvious issues
- **Complementary Analysis:** Use tool findings to guide deeper manual investigation
- **False Positive Filtering:** Apply human judgment to validate automated findings
- **Coverage Gaps:** Focus manual review on areas automated tools cannot effectively analyze

Security Metrics:

- **Manual Review Coverage:** Percentage of critical code paths reviewed by humans
- **Finding Quality:** Ratio of valid security issues to total findings
- **Review Efficiency:** Time spent on manual review vs. security value delivered
- **Trend Analysis:** Security posture improvement over time

Documentation Templates

Finding Report Template:

```
Title: [Vulnerability Type] in [Component]
Severity: [Critical/High/Medium/Low]
CWE: [CWE Number and Name]
Location: [File:Line or Function]
Description: [Detailed explanation of the vulnerability]
Impact: [Security implications and potential attack scenarios]
```

```

Reproduction: [Steps to reproduce or proof of concept]
Recommendation: [Specific fix guidance with code examples]
References: [CWE links, OWASP references, vendor documentation]
Status: [Open/In Progress/Fixed/Accepted Risk]
Assignee: [Developer responsible for fix]
Due Date: [Target fix date]

```

Review Summary Template:

```

Review Summary
=====
Application: [Application Name]
Version: [Version/Commit Hash]
Reviewer(s): [Names]
Review Date: [Date]
Scope: [Files/Components Reviewed]

Findings Summary:
- Critical: [Count]
- High: [Count]
- Medium: [Count]
- Low: [Count]
- Informational: [Count]

Key Recommendations:
1. [Priority recommendation]
2. [Priority recommendation]
3. [Priority recommendation]

Overall Risk Assessment: [Low/Medium/High/Critical]

```

Integration with SDLC

Review Timing

Baseline Review Integration

- **Project Initiation:** Comprehensive security assessment of existing codebase
- **Major Releases:** Full security review before significant version releases
- **Architecture Changes:** Complete review when fundamental design changes occur
- **Compliance Cycles:** Periodic comprehensive reviews for regulatory requirements
- **Security Incidents:** Thorough review following security breaches or major vulnerabilities
- **Onboarding Legacy Systems:** Initial security assessment when bringing existing applications under secure development practices

Diff-Based Review Integration

- **Pull Requests:** Security-focused review of code changes as part of standard PR process

- **Pre-commit Hooks:** Lightweight security checks on developer commits
- **Feature Completion:** Security review of completed user stories or features
- **Sprint Reviews:** Regular assessment of security implications of sprint deliverables
- **Hotfix Reviews:** Rapid security assessment of emergency fixes
- **Continuous Integration:** Automated triggering of security reviews based on code changes

Hybrid Approach

- **Risk-Based Scheduling:** Combine baseline reviews for high-risk components with diff-based reviews for routine changes
- **Incremental Baseline Updates:** Gradually expand baseline review coverage over multiple development cycles
- **Trigger-Based Reviews:** Escalate from diff-based to baseline review when significant security concerns are identified

For CI/CD integration and automated security testing, for more information see [CI CD Security Cheat Sheet](#).

Team Collaboration

Roles:

- **Security reviewers:** Conduct analysis and provide guidance
- **Developers:** Implement fixes and follow secure coding practices
- **Security champions:** Bridge security and development teams

Best Practices:

- Use standardized checklists and templates
- Maintain a knowledge base of common issues
- Track metrics on review effectiveness
- Provide regular security training
- Integrate with existing development workflows

Advanced Techniques

Race Condition Analysis

Focus on Time-of-Check vs Time-of-Use (TOCTOU) vulnerabilities and ensure atomic operations.

Business Logic Analysis

Analyze workflows for:

- State transitions and validation
- Opportunities to bypass steps or validation
- Proper validation at each workflow step
- Rollback mechanisms and cleanup on failures
- Behavior under concurrent access
- Boundary conditions and error scenarios

Security Architecture Review

Review architecture patterns for consistent security enforcement and proper API security controls.

Memory Safety

Review buffer management, integer overflow protection, and resource limits.

References

OWASP Resources:

- [OWASP Code Review Guide](#)
- [OWASP Testing Guide](#)
- [OWASP Top 10](#)
- [OWASP Application Security Verification Standard \(ASVS\)](#)
- [OWASP Secure Coding Practices](#)

Related OWASP Cheat Sheets:

- [Threat Modeling Cheat Sheet](#)
- [Abuse Case Cheat Sheet](#)
- [Attack Surface Analysis Cheat Sheet](#)
- [Secure Product Design Cheat Sheet](#)
- [Mass Assignment Cheat Sheet](#)
- [Insecure Direct Object Reference Prevention Cheat Sheet](#)
- [Cross-Site Request Forgery Prevention Cheat Sheet](#)

- [Server Side Request Forgery Prevention Cheat Sheet](#)
- [Unvalidated Redirects and Forwards Cheat Sheet](#)
- [Denial of Service Cheat Sheet](#)

Industry Standards:

- [CWE/SANS Top 25 Most Dangerous Software Errors](#)
- [NIST Secure Software Development Framework \(SSDF\)](#)
- [ISO/IEC 27034 - Application Security](#)

Additional Resources:

- [Microsoft Security Development Lifecycle \(SDL\)](#)
- [CERT Secure Coding Standards](#)